

Optimización de Caminos en Grafos de Gran Escala: Una Aplicación de Dijkstra y BFS en la Industria Cinematográfica

Resumen

El trabajo explora la interconectividad de la industria cinematográfica global a través de la Teoría de Grafos y algoritmos de búsqueda. No solo busca el camino más corto entre actores, sino que introduce funciones de costo variables (calidad cinematográfica y popularidad) para alterar la topología del grafo dinámicamente y crear un sistema de recomendación. Se detalla la ingeniería de datos necesaria para procesar *Big Data* en entornos de memoria limitada, la arquitectura de software web basada en Flask para la interacción en tiempo real y las estrategias de optimización para el despliegue en la nube, además de la implementación de algoritmos como BFS y Dijkstra, ambos bidireccionalmente.

1. Introducción y Definición del Problema

¿Hay un hilo invisible que conecta toda la industria cinematográfica? La teoría de los "Seis Grados de Separación" postula que cualquier persona en la Tierra puede estar conectada a cualquier otra a través de una cadena de no más de cinco intermediarios. En el contexto cinematográfico, esto se conoce como los "Grados de Bacon". Si bien los enfoques tradicionales suelen tratar todas las conexiones por igual (grafos no ponderados), el objetivo de esta investigación es trascender la búsqueda de caminos por mera proximidad (BFS) e implementar un sistema que valore la "calidad" del camino, permitiendo al usuario priorizar rutas a través de películas de alto *rating* o películas menos conocidas. Para hacer esto, se

aplicará el Algoritmo de Dijkstra sobre un grafo ponderado construido a partir de la base de datos de IMDb. Sin embargo, hay que tener en cuenta sus limitaciones, ya que IMDb sólo ofrece información sobre el reparto principal de las películas y no de todo el equipo y extras. Además, los datos fueron descargados el 16 de noviembre de 2025 y no fueron actualizados desde entonces.

2. Abstracción y Modelado Matemático

2.1. Definición Formal del Grafo

El sistema modela la industria cinematográfica como un Grafo Bipartito No Dirigido y Ponderado, denotado como $G = (U, V, E, w)$. En esta estructura el conjunto total de vértices ($U \cup V$) se particiona en dos subconjuntos disjuntos ($U \cap V = \emptyset$), donde 'U' representa a los actores (identificados por $nconst$) y 'V' representa a las películas (identificadas por $tconst$). La estructura de la topología queda definida por el conjunto de aristas (E), donde existe una relación $e = \{u, v\} \Leftrightarrow$ el actor $u \in U$ participó en la película $v \in V$. Esta configuración preserva la estructura bipartita, lo que implica que la relación no es reflexiva ni transitiva, dado que no existen adyacencia directas entre vértices de la misma clase, aunque sí es simétrica porque la conexión es no dirigida.

Modelo Bipartito $G = (U, V, E)$

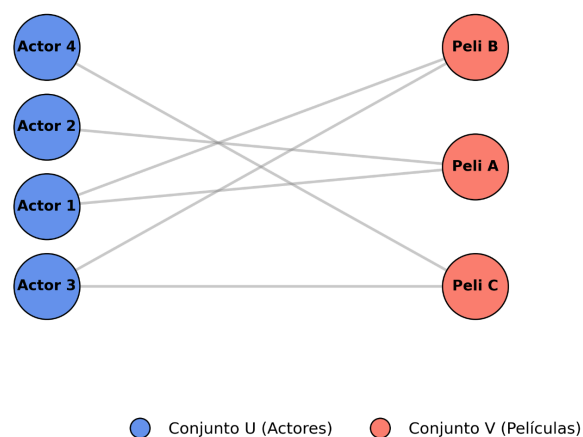


Figura 2.1. Visualización de la estructura bipartita

2.2. Funciones de Peso

A diferencia de un grafo estático, este proyecto implementa una función de peso ($w: E \Rightarrow \mathbb{R}^+$) dinámica, que varía según el perfil de búsqueda seleccionado por el usuario. Esto permite que el mismo conjunto de datos se comporte con topologías distintas dependiendo de la intención de la búsqueda.

1. Modo Casual (Ponderación Lineal):

Este perfil modela a un espectador promedio que busca evitar películas de mala calidad, pero que no penaliza la popularidad. La función de costo se define como una relación lineal inversa al *rating*:

$$w_{casual}(e) = 10.1 - Rating(v)$$

2. Modo Crítico (Ponderación Exponencial y Logarítmica):

Este perfil modela el comportamiento de un cinéfilo exigente que valora un cine más de nicho y rechaza activamente el cine comercial masivo, priorizando la excelencia sobre la proximidad y equilibrando el grafo hacia películas de alto culto. Para lograr este comportamiento no lineal, se diseñó una función de costo compuesta:

$$w_{crítico}(e) = FactorCalidad \times FactorFama$$

$$w_{crítico}(e) = 2,5^{10 - Rating(v)} \times (\log_{10}(Votos))^2$$

En primer lugar, el 'Factor de Calidad' abandona la progresión lineal del modo casual en favor de una función exponencial de base 2.5, donde el exponente está determinado por la distancia a la puntuación perfecta; esto implica que el costo de transitar por una arista aumenta drásticamente ante la más mínima caída en el *rating*. Simultáneamente, se

introduce un 'Factor de Fama', que encarece artificialmente las aristas de las producciones más famosas de IMDb, forzando al sistema a descartar los caminos populares y explorar rutas topológicas alternativas de menor exposición.

2.3. Implementación de los Algoritmos de Búsqueda

El sistema adopta un enfoque híbrido, seleccionando el algoritmo más eficiente según la naturaleza de la consulta (topológica vs. ponderada).

2.3.1. Búsqueda Bidireccional (Modo Velocidad)

Para las consultas donde el objetivo es minimizar la cantidad de intermediarios (aristas no ponderadas) se implementó una Búsqueda en Anchura (BFS) Bidireccional. El proceso inicia definiendo dos fronteras de exploración simultáneas: una desde el nodo origen *S* y otra desde el nodo destino *E*. En lugar de explorar el grafo como un círculo expansivo gigante desde el origen (lo que generaría un crecimiento exponencial de nodos visitados), se expanden dos círculos más pequeños hasta que se tocan.

La ejecución lógica del algoritmo se estructura en tres fases continuas. En la etapa de inicialización, se definen dos conjuntos de fronteras (*frontera_inicio* y *frontera_fin*) junto con sus respectivos diccionarios de procedencia, necesarios para la reconstrucción posterior del camino. El núcleo del proceso es un bucle de expansión alternada que se mantiene activo mientras ambas fronteras contengan elementos. En cada iteración, para optimizar el rendimiento, el algoritmo evalúa ambos conjuntos y selecciona aquel con menor cantidad de nodos para expandir; esta heurística de balanceo resulta crítica para evitar explorar la filmografía de actores con una extensa carrera. Finalmente, la condición de parada se activa mediante una verificación de intersección: si un nodo recién descubierto en la frontera activa ya se encuentra registrado en la frontera opuesta, se confirma el hallazgo del camino óptimo y el algoritmo detiene su ejecución de inmediato.

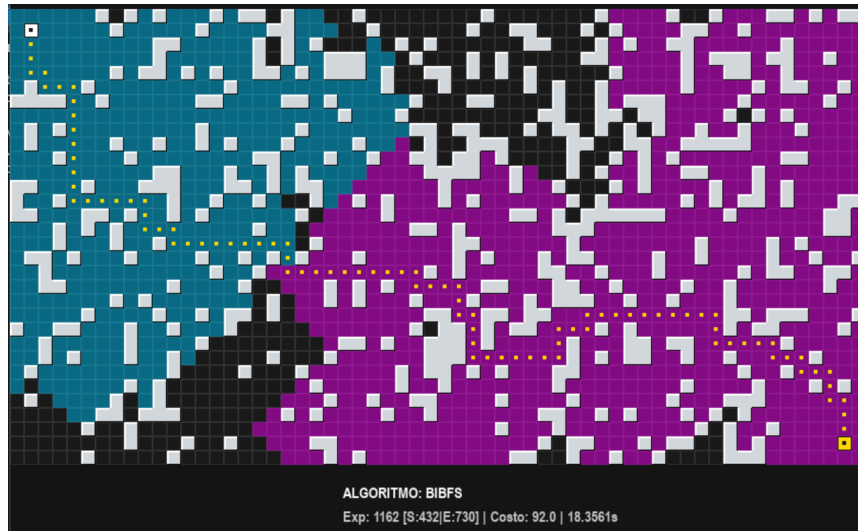


Figura 2.2. Abstracción visual del algoritmo BFS Bidireccional. La imagen ilustra conceptualmente cómo dos fronteras de búsqueda se expanden no uniformemente hasta intersectarse. **Nota:** Esta representación utiliza una grilla 2D para facilitar la visualización; la topología real del proyecto es un grafo bipartito no-euclidiano.

2.3.2. Algoritmo de Dijkstra (Modos Ponderados)

Para los perfiles "Casual" y "Crítico", donde las aristas poseen costos variables, se optó por una variante optimizada: el Algoritmo de Dijkstra Bidireccional. A diferencia del enfoque anterior no ponderado, este método prioriza la exploración de los caminos con menor costo acumulado mediante el uso de dos Colas de Prioridad, PQ_{Fwd} y PQ_{Rev} .

El proceso inicia estableciendo las distancias de los nodos de origen y destino en cero, asumiendo, pero no estableciendo, un valor infinito para el resto de los nodos hasta que son descubiertos. El núcleo del algoritmo sigue una estrategia de expansión balanceada similar a la búsqueda en anchura, iterando mientras existan candidatos en ambas direcciones. Sin embargo, la diferencia fundamental radica en el cambio de variables: en cada paso se extrae el nodo con la menor distancia acumulada de ambas fronteras, lo que garantiza que ese ya sea su costo mínimo, y se calcula una distancia tentativa para sus vecinos ($distancia_tentativa = distancia_actual + peso$). Si este

nuevo valor es menor al registrado previamente, se actualiza la prioridad del vecino en la estructura, garantizando así la expansión hacia el camino mínimo.

Finalmente, el criterio de parada es estricto. Aunque el algoritmo actualiza el mejor camino encontrado (μ) cada vez que se detecta una intersección entre las búsquedas, la ejecución no se detiene inmediatamente. Para asegurar el camino óptimo es necesario probar que no quedan caminos "ocultos" más cortos cruzando la zona no explorada. Esto se confirma sólo cuando se cumple la desigualdad:

$$Top(PQ_{Fwd}) + Top(PQ_{Rev}) \geq \mu$$

Esta condición actúa como una cota inferior matemática: la suma de los radios mínimos de ambas fronteras representa el menor costo posible para cualquier camino aún no descubierto. Al superar o igualar este valor al mejor camino ya hallado (μ), se demuestra que es imposible mejorar el resultado, permitiendo finalizar el proceso con total certeza.

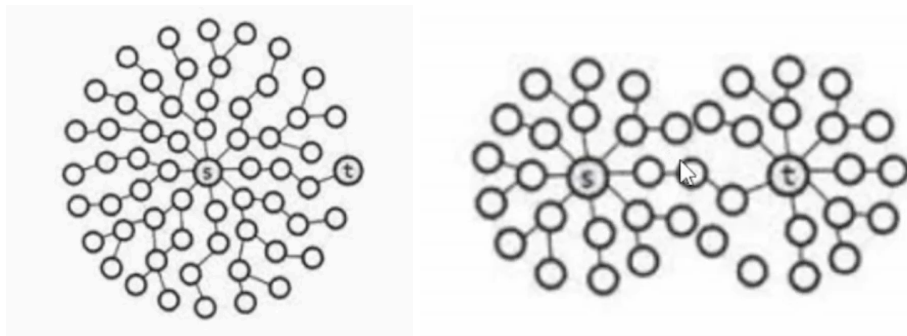


Figura 2.3. Visualización de la optimización de la bidireccionalidad.

2.4. Filtrado Topológico Dinámico

Para permitir la personalización de la búsqueda mediante criterios dinámicos (ej. "Películas de Acción de los 90"), se descartó la generación de subgrafos nuevos para cada filtro ya que, si bien esto aportaba información sobre la conectividad resultante de cada

filtro, el costo computacional lo hacía inviable ($O(|V| + |E|)$). En su lugar, se implementó una estrategia de 'Filtrado Topológico por Máscara'.

Este método define un conjunto de validación $S \subseteq V$, tal que todas las películas satisfacen los criterios de año, género, *rating* y votos.

$$S = \{v \in V / cumple_criterios(v)\}$$

Durante la fase de expansión del algoritmo, ya sea Dijkstra o BFS, se aplica una restricción lógica antes de añadir vértices a la frontera: debe verificar la pertenencia al conjunto. Esta técnica sustituye la costosa reconstrucción del grafo en memoria, que requiere iterar y copiar millones de datos, en una operación de verificación de pertenencia en un *set*, cuya complejidad es constante $O(1)$, lo que permite aplicar filtros complejos en tiempo real sin ralentizar el sistema.

3. Ingeniería de Datos y Estructuras en Memoria (ETL)

Se utilizaron cuatro fuentes de datos masivas de IMDb (*title.principals*, *title.basics*, *name.basics*, *title.ratings*) para llevar a cabo el proyecto.

3.1. Procesamiento por Lotes y Filtrado en Cascada

Para procesar los gigabytes de datos se diseñó un pipeline ETL (*Extract*, *Transform*, *Load*) que lee los archivos por lotes (*chunks*) de un millón de filas. El proceso restringe los datos exclusivamente a las categorías *movie* y *tvMovie*, descartando formatos ajenos al análisis como videojuegos o episodios de TV. Adicionalmente, aplica un filtro de calidad ($N > 100 \text{ votos}$) para garantizar la relevancia de los nodos y elimina las "islas", es decir, actores o películas que, tras los filtros anteriores, tienen grado cero.

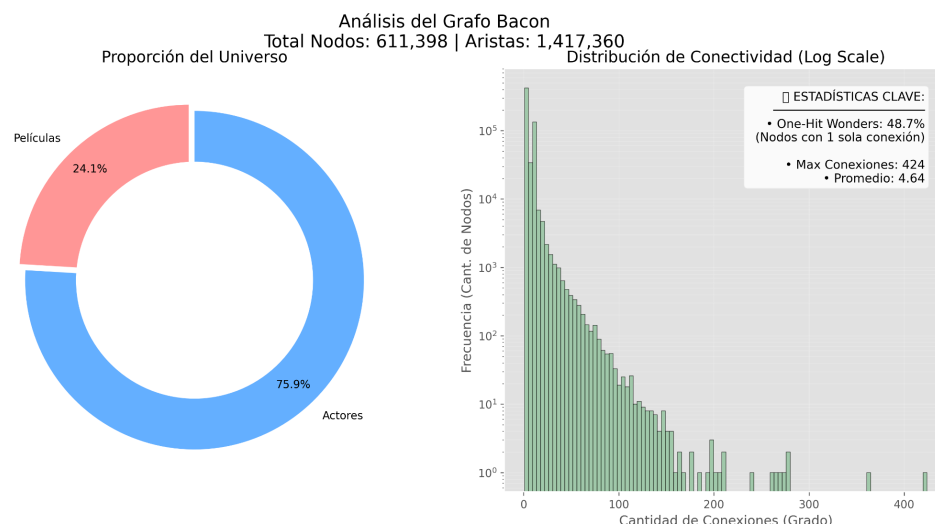


Figura 3.1. Visualización de la estructura del grafo.

3.2. Estructuras de Datos en Memoria (RAM)

La estructura central es un diccionario de listas de adyacencia (GRAFO) donde cada clave (ID de IMDb) apunta a una lista de nodos vecinos, permitiendo una navegación de complejidad $O(1)$. Dada la naturaleza bipartita, si la clave $u \in U$ (actor), su valor es una lista de $v \in V$ (películas) y viceversa. Paralelamente, se implementaron mapas auxiliares para el acceso directo a metadatos $O(1)$. *MAPA_PELICULAS* utiliza el ID *tconst* para obtener los valores para el cálculo de las funciones de costo y *MAPA_ACTORES* vincula los identificadores *nconst* con el nombre del actor. Finalmente, se desarrolló un índice invertido (*MAPA_NOMBRES*) que asocia cadenas de texto estandarizadas (sin acentos ni puntuación) con listas de IDs para las búsquedas del usuario.

3.3. Serialización

Dado el costo computacional de construir el grafo desde cero, se implementó un mecanismo de serialización binaria. El grafo procesado en el ETL se almacena en disco y se carga directamente a RAM, eliminando la latencia de reconstrucción.

4. Arquitectura de Software y Despliegue

El sistema se dividió en una arquitectura Cliente-Servidor desacoplada, donde la lógica de grafos reside en un entorno aislado en el Backend. JavaScript capta la solicitud del usuario y Python responde con el camino encontrado. El desafío principal consistió en trasladar la lógica teórica, validada en un entorno local con 16GB RAM, a un entorno de producción con restricciones estrictas (límite físico de 512MB). Esto obligó a bifurcar el proyecto en dos estrategias de gestión de memoria.

4.1. Deploy Branch y SQL

Para la versión de producción ("Deploy Branch"), se sustituyeron las estructuras en memoria por una base de datos relacional (SQLite). Si bien esto redujo drásticamente el consumo de la RAM, transformó el acceso a los vecinos de una operación en memoria inmediata a una consulta en disco (I/O Bound). En una nube de baja velocidad, esto resulta en errores de cuello de botella. Para mitigarlo, se implementó una estrategia híbrida con caché de atributos en RAM, precargando los valores necesarios para las funciones de costo (ratings y votos), lo que eliminó el 50% de las operaciones de lectura en disco. Finalmente, se identificó una limitación crítica en la recolección de datos: al usar SQLite, la información se guardaba en un archivo local dentro del mismo servidor. Debido a que los servidores gratuitos en la nube se reinician frecuentemente, este archivo se borraba y se perdía todo el historial acumulado. Para solucionar esto, se migró el almacenamiento de las estadísticas a una base de datos externa (PostgreSQL). Esto permite separar el procesamiento (el algoritmo que busca caminos) del almacenamiento (el historial de uso). De esta forma, aunque el servidor de la aplicación se reinicie o falle, los datos de las búsquedas permanecen seguros en un sistema independiente y permanente.

4.2. Escalabilidad Futura (Propuesta)

Se propone la implementación de Mapeo de Enteros (*Integer Mapping*). Actualmente, los identificadores alfanuméricos (ej. "nm0000102") consumen aprox. 50 bytes por objeto. Mapearlos a enteros primitivos de 4 bytes permitiría comprimir el grafo en arreglos contiguos de memoria, viabilizando el regreso a una arquitectura 100% en RAM incluso en entornos limitados. Asimismo, se plantea escalar hacia búsquedas heurísticas (como A^*), utilizando *clustering* de nodos para guiar la expansión del algoritmo de manera inteligente y poder crear nuevos perfiles basados en los clusters.

5. Validación Experimental y Análisis Estadístico

Para trascender la implementación teórica y verificar la eficacia del sistema en un entorno de grafos masivos, se diseñó y ejecutó un experimento de dos fases: un *Benchmark* de Rendimiento Algorítmico para medir la eficiencia computacional y una Simulación de Montecarlo para analizar el funcionamiento individual de cada modo, además de comparar los resultados con la centralidad teórica del grafo.

5.1. Metodología del Experimento

El experimento se realizó sobre el subgrafo conexo principal de la industria, filtrando actores con un grado de conexión $d \geq 15$ para garantizar búsquedas significativas. Se ejecutaron más de 200.000 simulaciones de búsquedas de caminos entre pares de actores seleccionados aleatoriamente ($u, v \in U$) utilizando la versión del sistema en memoria (RAM) para acelerar el proceso. Finalmente, se trabajó con más de un 1,7 millones de filas, que pertenecen a los actores o películas a los que se visitó en cada búsqueda.

5.2. Resultados de Eficiencia Algorítmica (Benchmark)

En el "Modo Velocidad" (grafos no ponderados), la diferencia de rendimiento validó empíricamente la reducción del espacio de búsqueda de $O(b^d)$ a $O(B^{d/2})$. Los resultados mostraron una disparidad drástica en la cantidad de nodos visitados: mientras que el enfoque unidireccional requirió explorar un promedio de 136,000 nodos para encontrar una conexión de tres o cuatro grados, la implementación bidireccional convergió visitando apenas 1,400 nodos. Esta reducción del espacio de búsqueda se tradujo en una aceleración promedio de 78 veces, logrando disminuir los tiempos de respuesta de aproximadamente 0.9 segundos a tan solo 0.01 segundos, lo cual confirma la inviabilidad del enfoque tradicional para interacciones en tiempo real. Para los modos ponderados ("Casual" y "Crítico") la aceleración fue menor ya que es un proceso más riguroso, aunque representó una significativa disminución de 6,5 en promedio respecto a la búsqueda unidireccional.

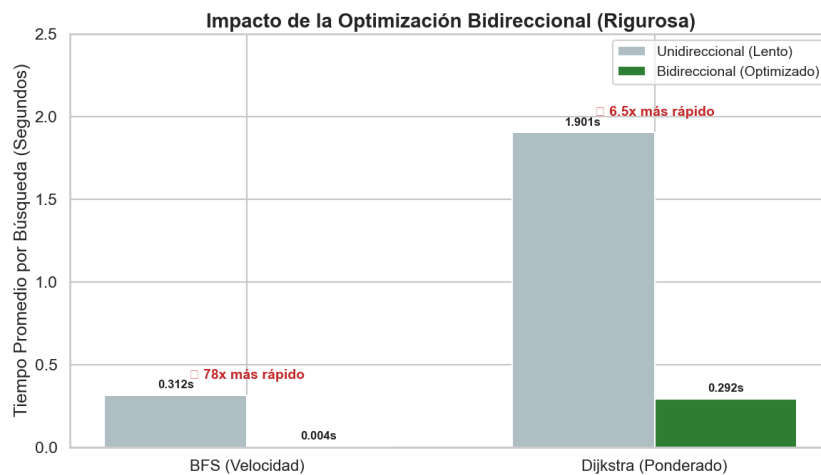


Figura 5.1. Comparativa de rendimiento computacional. Se observa la reducción drástica del tiempo de ejecución gracias a la estrategia bidireccional, logrando una aceleración de 78x en búsquedas no ponderadas y 6.5x en grafos ponderados.

5.3. Análisis de Redes y Tendencias (Resultados de Montecarlo)

Tras procesar más de 100,000 caminos óptimos, se analizaron las características de las películas que actúan como "puentes" en el grafo, revelando cómo las funciones de costo alteran la topología de la red.

5.3.1. Distribución de Calidad (Ratings)

Los datos obtenidos confirman que la función de costo del "Modo Crítico" penaliza exitosamente el contenido de baja calidad. Mientras el "Modo Velocidad" presenta una dispersión de ratings amplia con una mediana cercana a 6,5, utilizando cualquier película disponible para cerrar el camino, el "Modo Crítico" desplaza la mediana significativamente hacia 7,5 y reduce la varianza. En el "Modo Casual" sucede algo similar, solo que aumenta la varianza. Esto indica que el algoritmo prefiere desviarse por caminos topológicamente más largos si estos transcurren a través de nodos de alta calidad.

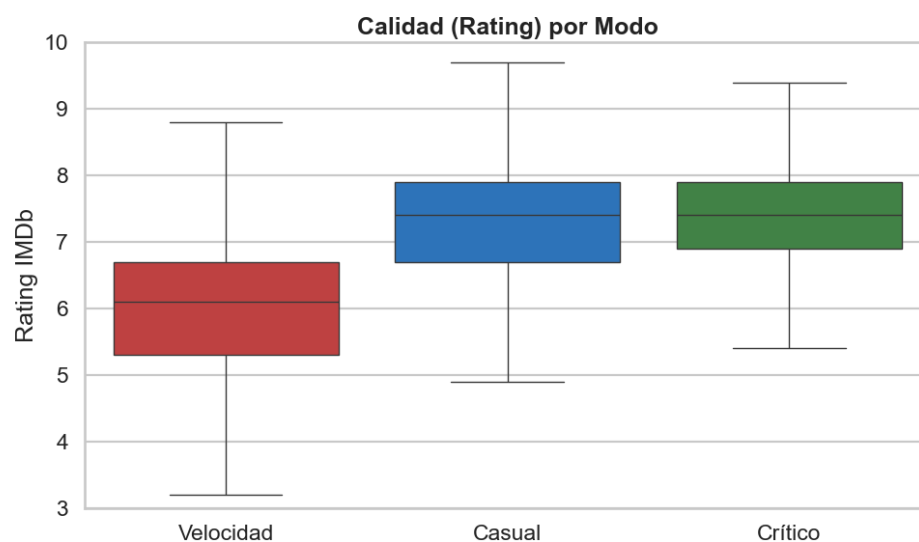


Figura 5.2. Diagrama de caja (*Boxplot*) de los *ratings* promedio por camino. El "Modo Crítico" (verde) desplaza significativamente la mediana hacia calificaciones superiores, filtrando el contenido de baja calidad presente en el "Modo Velocidad".

5.3.2. Sesgo Temporal (Años)

Se observó también un comportamiento emergente respecto a las eras cinematográficas seleccionadas por cada algoritmo. El "Modo Velocidad" muestra una preferencia marcada por el cine contemporáneo (2000-2024). Este fenómeno se correlaciona con el crecimiento exponencial del volumen de producción y la densidad de actores activos en la industria moderna, lo que aumenta la probabilidad estadística de encontrar conexiones recientes. En contraste, el "Modo Crítico" aplanan la curva temporal, recuperando una mayor densidad de películas de las décadas de 1970 y 1990. Esto valida la capacidad del sistema para redescubrir "Clásicos de Culto" que actúan como puentes de calidad entre generaciones, evitando el sesgo de actualidad que si tiene el "Modo Casual".

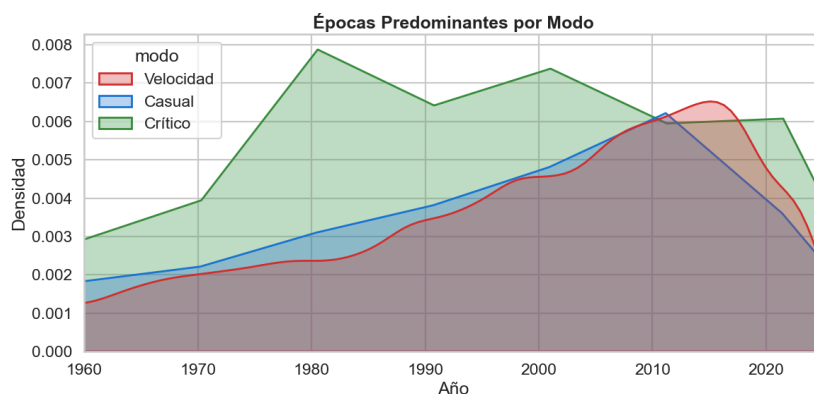


Figura 5.3. Densidad temporal de las películas seleccionadas. Nótese cómo el "Modo Velocidad" (rojo) y "Modo Casual" (azul) presentan un sesgo de actualidad marcado hacia el cine post-2000, mientras que el "Modo Crítico" (verde) distribuye la navegación más equitativamente a lo largo de las décadas.

5.3.3. Centralidad de Intermediación

Finalmente, el experimento permitió disociar la Centralidad de Grado de la Centralidad de Intermediación. Si bien un análisis estático señala a las películas de animación como los nodos de mayor grado debido a la magnitud de sus elencos de voz, la simulación de Montecarlo reveló que estas no actúan como puentes efectivos en la navegación real del grafo. Esto demuestra que la mera acumulación de aristas no garantiza relevancia topológica si los actores conectados no son relevantes en la topología del grafo.

Al analizar la Centralidad de Intermediación, los verdaderos conectores emergen según la intención de búsqueda: En el modo 'Casual' el sistema prioriza el *Canon Popular* (ej. *The Godfather*, *Inception*), indicando que las producciones que tienen tanto alta aceptación crítica como del público sirven como puentes entre diversas comunidades de actores. Por otro lado, en el modo 'Crítico' se observa una tendencia hacia documentales sobre actores (Henry Fonda: The Man and His Movies); anomalías con bajo rating, pocos votos y reparto famoso ("As You Like It") y joyas ocultas con pocos votos (My Hero). Esto estaría demostrando que, al penalizar la popularidad, el modo 'Crítico' favorece nichos extremos, lo cual sugiere que la función de costo podría refinarse para equilibrar mejor la relación 'calidad' y 'oscuridad'.

□ Top 10 Películas "Puede" según el Modo

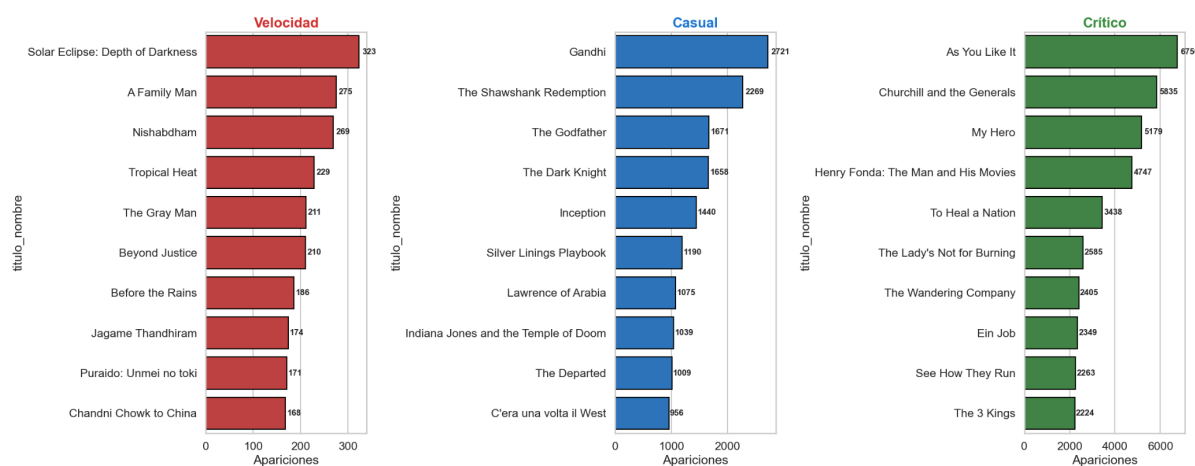


Figura 5.4. Top 10 de películas con mayor centralidad de intermediación (*Betweenness Centrality*) según el perfil de búsqueda. Se evidencia cómo cada algoritmo prioriza conjuntos de películas disjuntos.

Es importante notar que la simulación de Montecarlo asume una distribución uniforme de consultas. En contraste, el comportamiento orgánico del usuario no es aleatorio, sino que exhibe un sesgo hacia tendencias culturales y temporales. En consecuencia, el despliegue en producción no actúa solo como un buscador, sino como un instrumento de observabilidad. La validación definitiva del modelo trasciende la simulación: Con la recolección de datos se podrá analizar en tiempo real la convergencia, o divergencia, de los nodos que la teoría matemática señala como centrales con los puentes reales que utilizan los usuarios humanos.

5.4. Conclusión del Experimento

La experimentación valida la hipótesis central del trabajo: es posible alterar la percepción de la distancia en una red social mediante la manipulación matemática de los pesos de las aristas. El sistema no solo encuentra conexiones, sino que las cura activamente, ofreciendo rutas que son estadísticamente distintas y semánticamente coherentes con el perfil del usuario (Velocidad vs. Calidad).

6. Conclusión

Si bien se suele decir que la matemática es pura abstracción, este trabajo no busca profundizar en ella sino, todo lo contrario, plasmar la teoría en lo concreto. A través de la lógica computacional y la teoría de grafos, se logró trazar un esquema capaz de explorar millones de conexiones cinematográficas, transformando datos crudos en un mapa cultural navegable.

El desarrollo del proyecto trascendió la mera implementación de algoritmos. Implicó la estructuración de relaciones simétricas y la definición de conjuntos lógicos para dar forma a una idea. Fue un proceso iterativo de ingeniería: desde la limpieza de *Big Data* para garantizar la eficiencia, hasta la superación de las barreras físicas de la computación en la nube, donde el sistema se vio forzado a escalar, fallar y ser reconstruido bajo nuevas estrategias. Así, se demostró que la exploración de grafos masivos no depende tanto de la potencia del hardware, sino de la precisión del modelo matemático empleado.

Originalmente, el problema se planteaba como la búsqueda de un "hilo invisible" que uniera a toda la industria. No obstante, en una era digital donde el volumen de información (y generación) es infinito y las herramientas están al alcance de todos, la mera existencia de un camino entre dos puntos ha dejado de ser la incógnita principal. Los resultados de la simulación de Montecarlo revelaron que no existe una única industria cinematográfica, sino múltiples topologías superpuestas: nombramos a los *blockbusters* masivos y las obras de culto, pero existen tantos como hay gustos. La verdadera pregunta ya no es *si* existe una conexión, sino *cómo* podemos utilizar nuestro conocimiento para hallar, dentro del ruido de los datos masivos y de todos los caminos posibles, uno que sea relevante.